

VLSI INTERNSHIP – TASK 5

VERILOG HDL IMPLEMENTATION OF DATAPATH, ALU, AND
MEMORY COMPONENTS (RAM/ROM)

Submitted By : Likhith Gowda H R

Department of Electronics and Communication Engineering

Dayananda Sagar Academy of Technology and Management (DSATM)

Internship Domain : VLSI Design

Organization : Maincrafts Technology

July 2026

1. Introduction

Verilog HDL (Hardware Description Language) is widely used for designing, modeling, and verifying digital hardware systems in FPGA and ASIC design flows. Datapath components such as the Arithmetic Logic Unit (ALU), Register File, Random Access Memory (RAM), and Read-Only Memory (ROM) are fundamental building blocks of modern digital processors and embedded systems. These components perform arithmetic and logic operations, temporary data storage, memory access, and data transfer within digital systems.

In this task, an **Arithmetic Logic Unit (ALU)**, **Register File**, **RAM**, **ROM**, and a **simple Datapath** were implemented using Verilog HDL. Functional verification was carried out using Icarus Verilog, and the generated waveforms were analyzed using GTKWave to validate the correctness of each design.

1.1 Objectives

- To understand the design of datapath components using Verilog HDL.
- To implement an Arithmetic Logic Unit (ALU).
- To design a Register File with read and write operations.
- To implement Random Access Memory (RAM).
- To implement Read-Only Memory (ROM).
- To integrate the ALU into a simple Datapath.
- To simulate the designs using Icarus Verilog.
- To analyze simulation waveforms using GTKWave.

1.2 Tools Used

- Verilog HDL
- Icarus Verilog
- GTKWave
- Ubuntu (WSL)

2. Arithmetic Logic Unit (ALU) Implementation

2.1 Verilog Code

The following Verilog HDL code implements a 4-bit Arithmetic Logic Unit (ALU). The ALU performs arithmetic and logical operations such as addition, subtraction, AND, OR, and XOR based on the select signal.

alu.v

```
module alu(  
    input [3:0] A,  
    input [3:0] B,  
    input [2:0] sel,  
    output reg [3:0] Y  
);  
  
always @(*) begin  
    case(sel)  
        3'b000: Y = A + B;    // Addition  
        3'b001: Y = A - B;    // Subtraction  
        3'b010: Y = A & B;    // AND  
        3'b011: Y = A | B;    // OR  
        3'b100: Y = A ^ B;    // XOR  
        default: Y = 4'b0000;  
    endcase  
end  
  
endmodule
```

2.2 Testbench Code

The following testbench verifies the functionality of the Arithmetic Logic Unit by applying different input combinations and select signals to perform various arithmetic and logical operations.

tb_alu.v

```
`timescale 1ns/1ps

module tb_alu;

    reg [3:0] A;

    reg [3:0] B;

    reg [2:0] sel;

    wire [3:0] Y;

    alu uut (

        .A(A),

        .B(B),

        .sel(sel),

        .Y(Y)

    );

    initial begin

        $dumpfile("alu.vcd");

        $dumpvars(0, tb_alu);

        A = 4'b0101; B = 4'b0011;

        sel = 3'b000; #10;    // Addition

        sel = 3'b001; #10;    // Subtraction

        sel = 3'b010; #10;    // AND
```

```

        sel = 3'b011; #10;    // OR

        sel = 3'b100; #10;    // XOR

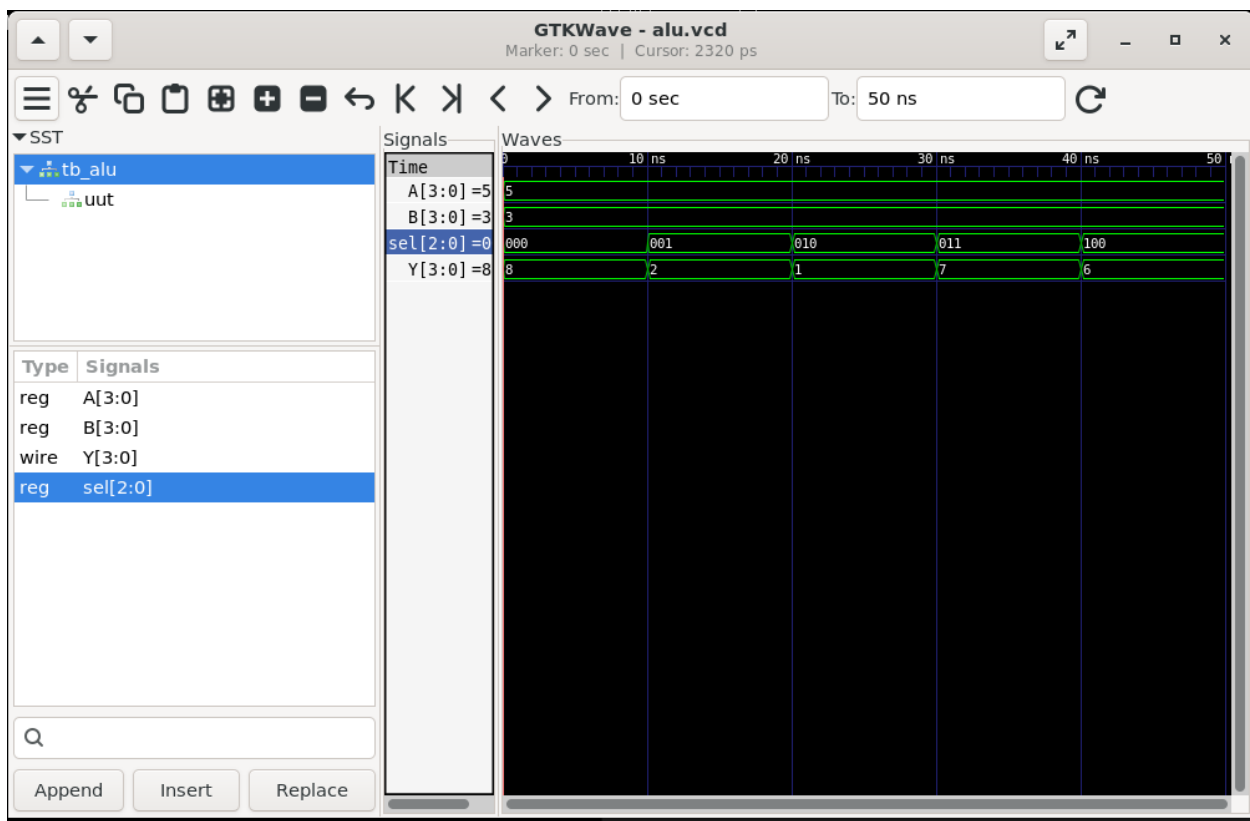
    $finish;

end

endmodule

```

2.3 Waveform Output



Observation:

The ALU was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the ALU correctly performs addition, subtraction, AND, OR, and XOR operations based on the select signal. The output matches the expected result for each arithmetic and logical operation, validating the correct functionality of the ALU implementation.

3. Register File Implementation

3.1 Verilog Code

The following Verilog HDL code implements a 4-register Register File. The Register File supports synchronous write operations on the positive edge of the clock and asynchronous read operations based on the selected read address.

register_file.v

```
module register_file(  
  
    input clk,  
  
    input we,  
  
    input [1:0] wr_addr,  
  
    input [1:0] rd_addr,  
  
    input [3:0] data_in,  
  
    output reg [3:0] data_out  
  
);  
  
// 4 Registers of 4 bits each  
  
reg [3:0] reg_file [3:0];  
  
integer i;  
  
// Initialize registers to zero  
  
initial begin
```

```
        for(i = 0; i < 4; i = i + 1)

            reg_file[i] = 4'b0000;

    end

    // Write operation

    always @(posedge clk) begin

        if (we)

            reg_file[wr_addr] <= data_in;

    end

    // Read operation

    always @(*) begin

        data_out = reg_file[rd_addr];

    end

endmodule
```

3.2 Testbench Code

The following testbench verifies the read and write operations of the Register File by applying different write addresses, read addresses, and input data values while generating the required clock signal.

tb_register_file.v

```
`timescale 1ns/1ps

module tb_register_file;

    reg clk;

    reg we;

    reg [1:0] wr_addr;

    reg [1:0] rd_addr;

    reg [3:0] data_in;

    wire [3:0] data_out;

    register_file uut (

        .clk(clk),

        .we(we),

        .wr_addr(wr_addr),

        .rd_addr(rd_addr),

        .data_in(data_in),

        .data_out(data_out)

    );

    // Clock generation

    always #5 clk = ~clk;

    initial begin

        $dumpfile("register_file.vcd");

        $dumpvars(0, tb_register_file);
```



```

    clk = 0;

    we = 1;

    wr_addr = 2'b00;

    rd_addr = 2'b00;

    data_in = 4'b0000;

    // Write 5

    wr_addr = 2'b00;

    data_in = 4'b0101;

    #10;

    // Write 10 into Register 1

    wr_addr = 2'b01;

    data_in = 4'b1010;

    #10;

    // Read Register 0

    we = 0;

    rd_addr = 2'b00;

    #10;

    // Read Register 1

    rd_addr = 2'b01;

    #10;

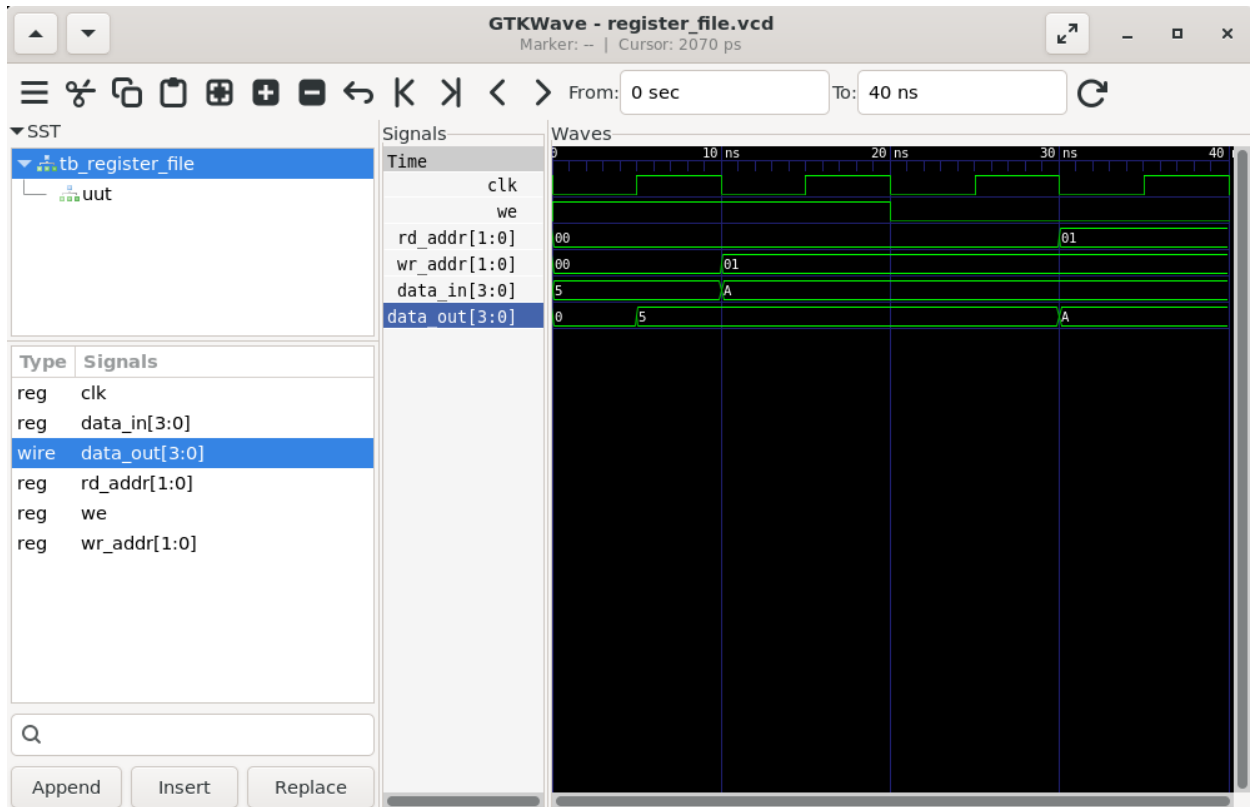
    $finish;

end

endmodule

```

3.3 Waveform Output



Observation:

The Register File was successfully simulated using Icarus Verilog and GTKWave. The waveform demonstrates the correct write and read operations of the Register File. Data is written into the selected register on the positive edge of the clock when the write enable (we) signal is high and is correctly read from the selected register using the read address (rd_addr), validating the correct functionality of the Register File.

4. RAM Implementation

4.1 Verilog Code

The following Verilog HDL code implements a 4×4 Random Access Memory (RAM). The RAM supports synchronous write operations on the positive edge of the clock and asynchronous read operations based on the selected memory address.

ram.v

```
module ram(  
    input clk,  
    input we,  
    input [1:0] addr,  
    input [3:0] data_in,  
    output reg [3:0] data_out  
);  
  
reg [3:0] memory [3:0];  
  
integer i;  
  
// Initialize RAM  
initial begin  
    for(i = 0; i < 4; i = i + 1)  
        memory[i] = 4'b0000;  
end  
  
// Write operation  
always @(posedge clk) begin  
    if (we)  
        memory[addr] <= data_in;  
end
```

```

end

// Read operation

always @(*) begin

    data_out = memory[addr];

end

endmodule

```

4.2 Testbench Code

The following testbench verifies the write and read operations of the RAM by applying different memory addresses and input data values while generating the required clock signal.

tb_ram.v

```

`timescale 1ns/1ps

module tb_ram;

    reg clk;
    reg we;
    reg [1:0] addr;
    reg [3:0] data_in;
    wire [3:0] data_out;

    ram uut (
        .clk(clk),
        .we(we),
        .addr(addr),
        .data_in(data_in),
        .data_out(data_out)
    )

```

```
);
```

```
// Clock generation
```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
    $dumpfile("ram.vcd");
```

```
    $dumpvars(0, tb_ram);
```

```
    clk = 0;
```

```
    we = 1;
```

```
    // Write 5 to address 0
```

```
    addr = 2'b00;
```

```
    data_in = 4'b0101;
```

```
    #10;
```

```
    // Write A to address 1
```

```
    addr = 2'b01;
```

```
    data_in = 4'b1010;
```

```
    #10;
```

```
    // Read address 0
```

```
    we = 0;
```

```
    addr = 2'b00;
```

```
    #10;
```

```
    // Read address 1
```

```
    addr = 2'b01;
```

```

        #10;

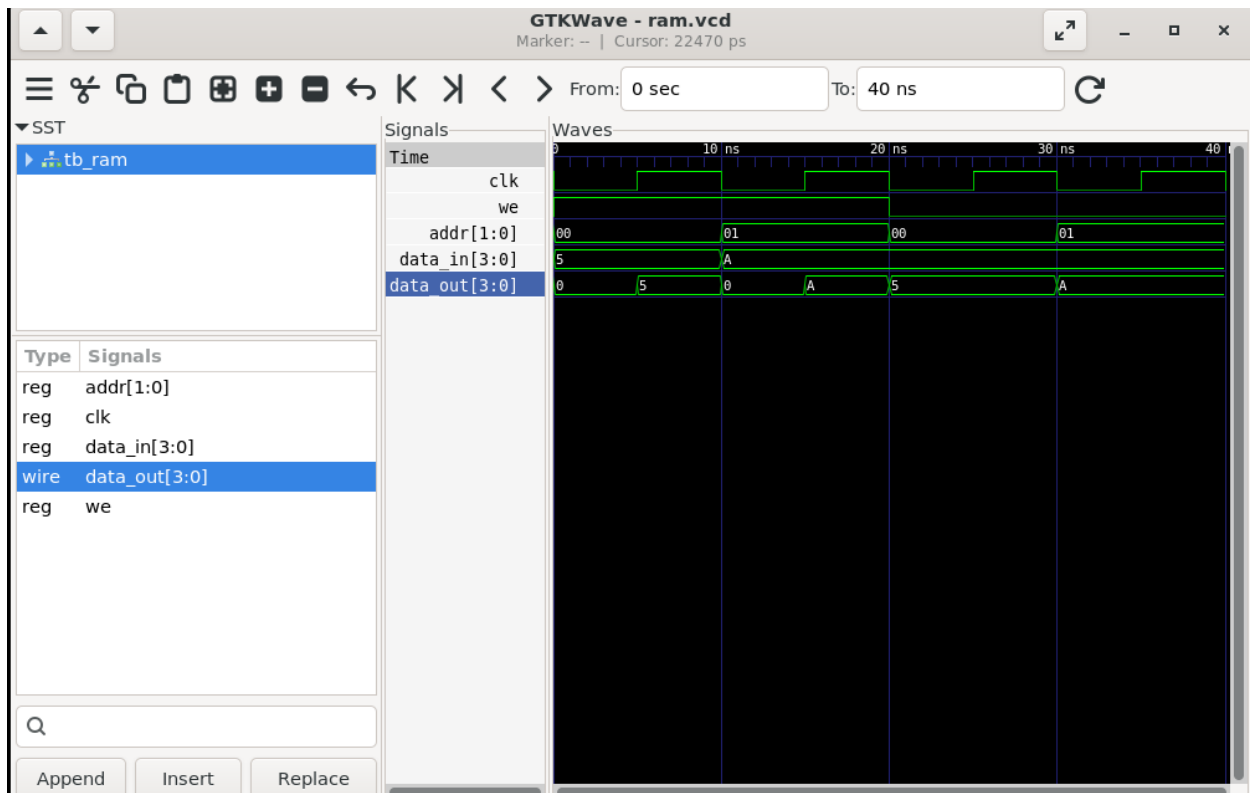
        $finish;

end

endmodule

```

4.3 Waveform Output



Observation:

The RAM was successfully simulated using Icarus Verilog and GTKWave. The waveform verifies that data is correctly written into the selected memory locations on the positive edge of the clock when the write enable (we) signal is high. When the write enable is low, the stored data is successfully read from the selected memory address, confirming correct RAM read and write functionality.

5. ROM Implementation

5.1 Verilog Code

The following Verilog HDL code implements a 4×4 Read-Only Memory (ROM). The ROM stores predefined data values, and the output changes according to the selected address without requiring any write operation.

rom.v

```
module rom(  
    input [1:0] addr,  
    output reg [3:0] data_out  
);  
  
always @(*) begin  
    case(addr)  
        2'b00: data_out = 4'b0001;  
        2'b01: data_out = 4'b0010;  
        2'b10: data_out = 4'b0100;  
        2'b11: data_out = 4'b1000;  
        default: data_out = 4'b0000;  
    endcase  
end  
  
endmodule
```

5.2 Testbench Code

The following testbench verifies the functionality of the ROM by applying different address values and observing the corresponding stored data at the output.

tb_rom.v

```
`timescale 1ns/1ps

module tb_rom;

reg [1:0] addr;

wire [3:0] data_out;

rom uut (

    .addr(addr),

    .data_out(data_out)

);

initial begin

    $dumpfile("rom.vcd");

    $dumpvars(0, tb_rom);

    addr = 2'b00; #10;

    addr = 2'b01; #10;

    addr = 2'b10; #10;

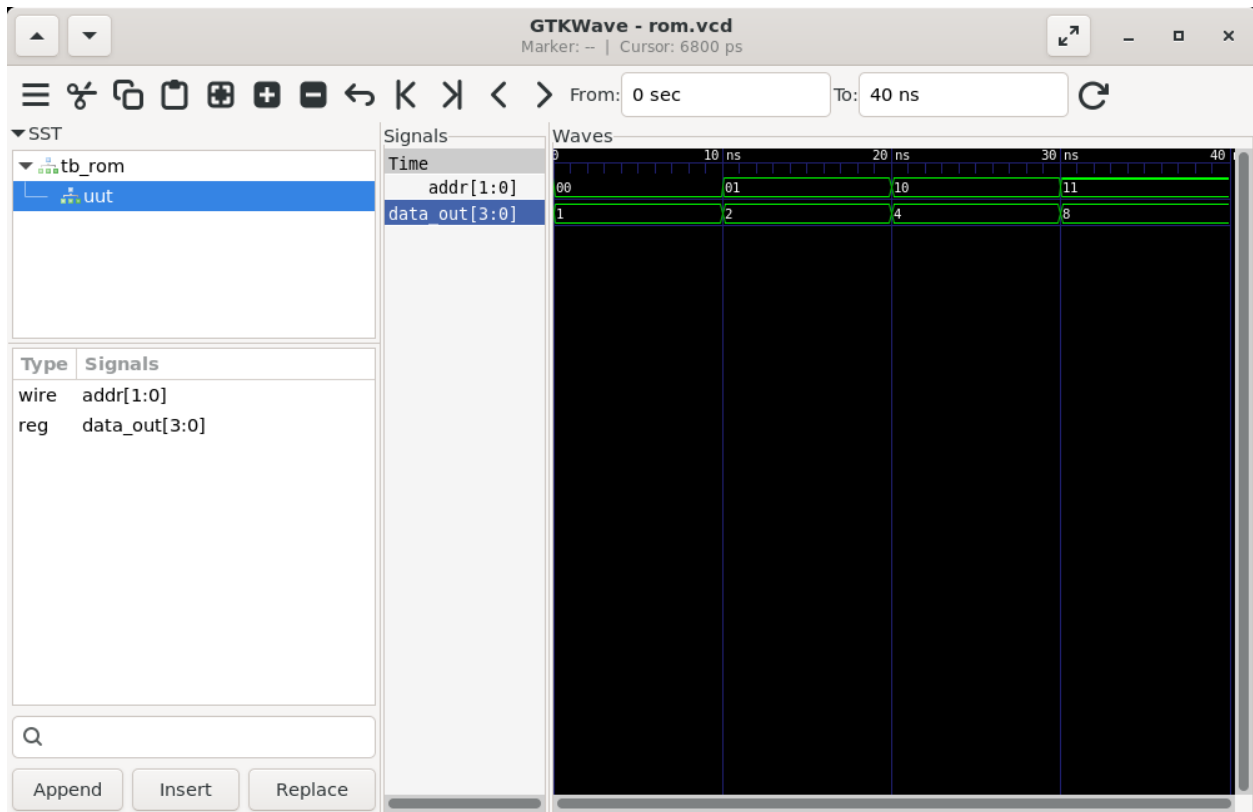
    addr = 2'b11; #10;

    $finish;

end

endmodule
```


5.3 Waveform Output



Observation:

The ROM was successfully simulated using Icarus Verilog and GTKWave. The waveform verifies that the predefined data values are correctly produced for each input address. Since ROM is a read-only memory, the stored data remains constant and changes only according to the selected address, confirming the correct functionality of the ROM implementation.

6. Datapath Implementation

6.1 Verilog Code

The following Verilog HDL code implements a simple datapath by integrating the Arithmetic Logic Unit (ALU). The datapath accepts two input operands and a select signal, forwards the inputs to the ALU, and produces the corresponding arithmetic or logical output. This demonstrates the basic flow of data within a digital processing system.

datapath.v

```
module datapath(  
    input [3:0] A,  
    input [3:0] B,  
    input [2:0] sel,  
    output [3:0] result  
);  
  
    alu u1 (  
        .A(A),  
        .B(B),  
        .sel(sel),  
        .Y(result)  
    );  
  
endmodule
```

6.2 Testbench Code

The following testbench verifies the operation of the datapath by applying different input values and select signals. The generated output is observed to ensure that the datapath correctly performs arithmetic and logical operations through the integrated ALU.

tb_datapath.v

```
`timescale 1ns/1ps

module tb_datapath;

    reg [3:0] A;
    reg [3:0] B;
    reg [2:0] sel;
    wire [3:0] result;
    datapath uut (
        .A(A),
        .B(B),
        .sel(sel),
        .result(result)
    );
    initial begin
        $dumpfile("datapath.vcd");
        $dumpvars(0, tb_datapath);

        A = 4'b0101;    // 5
        B = 4'b0011;    // 3

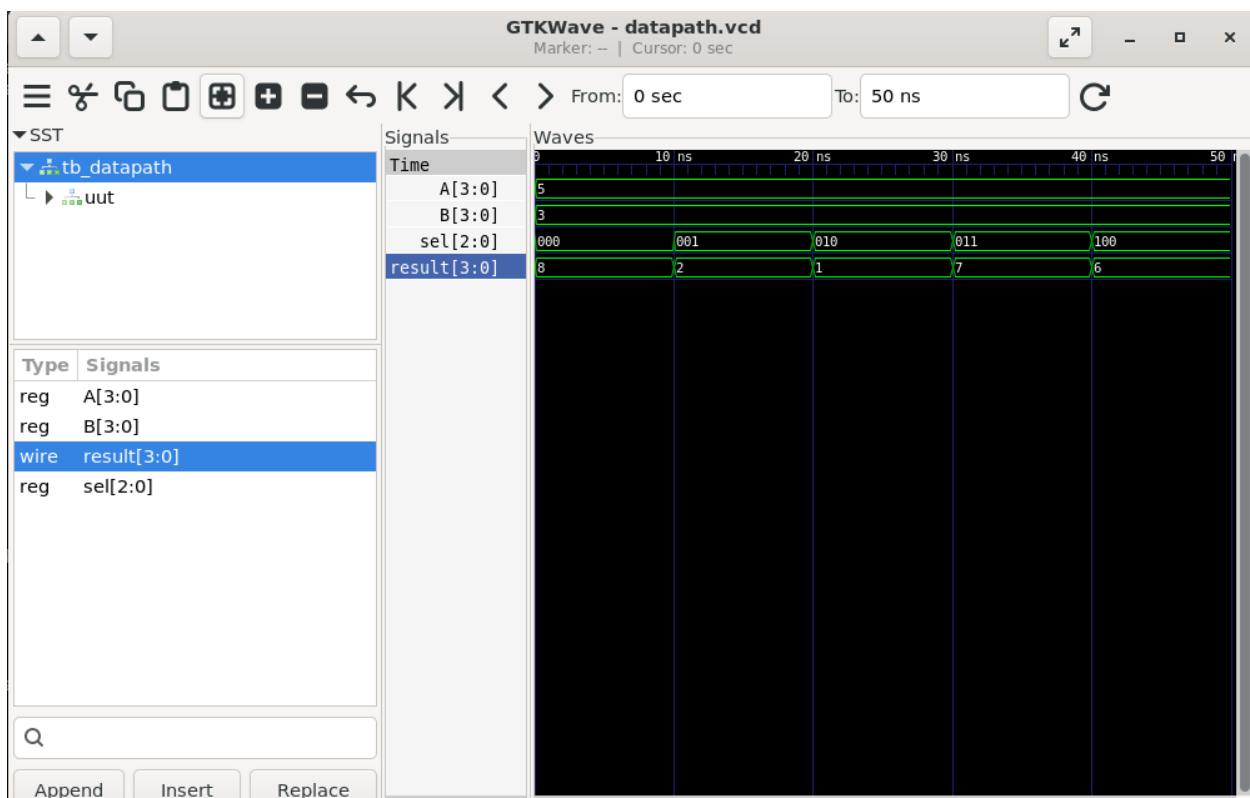
        sel = 3'b000; #10;    // ADD
        sel = 3'b001; #10;    // SUB
```

```

        sel = 3'b010; #10;    // AND
        sel = 3'b011; #10;    // OR
        sel = 3'b100; #10;    // XOR
        $finish;
    end
endmodule

```

6.3 Waveform Output



Observation:

The datapath was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the datapath correctly transfers the input operands to the ALU and performs arithmetic and logical operations according to the select signal. The output matches the expected results, validating the correct integration of the datapath components.

7. Results

The Arithmetic Logic Unit (ALU), Register File, RAM, ROM, and Datapath were successfully implemented using Verilog HDL. Functional verification was carried out using dedicated testbenches, and waveform analysis using GTKWave confirmed that all modules performed according to their expected functionality. The successful integration of these components demonstrates the fundamental operation of datapath and memory elements used in digital systems and VLSI design.

8. Applications

- FPGA Design
- ASIC Design
- Processor Datapath Design
- Embedded Systems
- Digital Signal Processing
- Computer Architecture
- Memory Systems
- VLSI Design and Verification

9. Conclusion

The ALU, Register File, RAM, ROM, and Datapath were successfully implemented and verified using Verilog HDL. Simulation with Icarus Verilog and waveform analysis using GTKWave confirmed the correct functionality of all modules, providing practical experience in digital design and VLSI system development.

■ References

1. Icarus Verilog Documentation
2. GTKWave User Guide
3. Verilog HDL – Samir Palnitkar
4. Digital Design – Morris Mano
5. IEEE Verilog HDL Standard